

A VISUAL GUIDE

Crystal Graph Neural Networks

Predicting elastic moduli from crystal structure – a from-scratch reproduction of PINK



Aizaz Khalid · reproduction of PINK, J. Mater. Inf. 2025, 5, 12

Why predict elastic moduli at all?

The bottleneck

Elastic moduli come from DFT calculations costing hours to days of compute – only $\sim 11,000$ of the Materials Project's $\sim 150,000$ entries have them.

Screening a large candidate space by DFT alone is not affordable.

Why they matter here

PINK screens for materials with ultralow *lattice thermal conductivity* κ_L , whose physics stage needs bulk modulus K and shear modulus G as inputs.

Predicting K and G in milliseconds turns an impossible screen into a routine one.

A CIF file says where the atoms are. It does not say how stiff the crystal is – that is what we are learning to predict.

Why not just a standard neural network?

A dense network expects a fixed-length vector; a CNN expects a fixed-size grid. Neither fits a crystal:

Variable size

A unit cell can hold 2 atoms or 200 – no fixed vector length to build a dense layer around.

No grid

Atoms sit at arbitrary 3D positions, not on the regular pixel grid a CNN's kernel slides across.

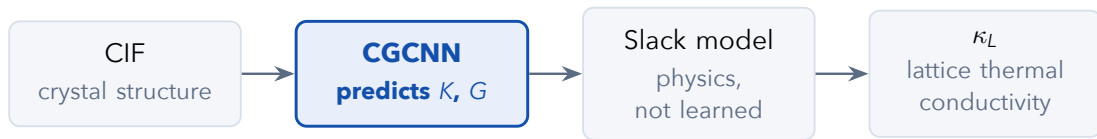
It repeats forever

A crystal is periodic – tiling space in three directions, not croppable to a fixed patch.

The fix: represent it as a *graph* – atoms as nodes, bonds as edges – which natively handles a variable number of objects in arbitrary positions. A graph neural network passes messages along the edges instead of convolving over pixels; CGCNN is exactly this, built for crystals.

Everything from here on is about the two things a graph needs: a node, and an edge.

Where this project fits inside PINK



The elastic moduli are the *only* learned quantity. Everything downstream of them is physics, not machine learning.

This project implements the first arrow – $\text{CIF} \rightarrow (K, G)$ – including the network itself, written from scratch rather than forked from the paper's code. Its output is the (K, G) table the κ_L stage consumes.

Scope of this talk: the machine-learning half of PINK, explained from first principles and reproduced independently.

PART TWO

From Crystal to Graph

Nodes, edges, and a worked example in real sodium chloride

Meet the crystal: rock-salt NaCl

Ordinary table salt. Its structure – **rock salt** – is one of the simplest in nature, and every number here comes from the actual CIF file used throughout this talk.

Cubic cell, side $a = 5.62 \text{ \AA}$

8 ions per cell: 4 Na^+ , 4 Cl^- .

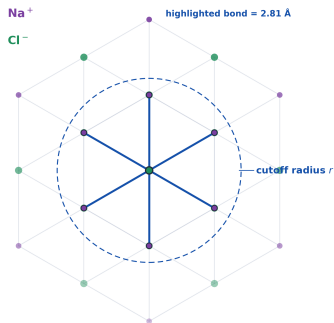
Every Na^+ sits surrounded by 6 Cl^- neighbours, all at exactly $a/2 = 2.81 \text{ \AA}$ – the real crystal's coordination, not a simplification.

Fractional coordinates, from the CIF:

ion	x	y	z
Na^+	0	0	0
Na^+	0	0.5	0.5
Na^+	0.5	0	0.5
Na^+	0.5	0.5	0
Cl^-	0.5	0.5	0.5
Cl^-	0.5	0	0
Cl^-	0	0.5	0
Cl^-	0	0	0.5

A CIF is nothing more than a table like this one, plus the cell size and shape. Turning it into a graph is the next three slides.

Nodes, edges, and a cutoff radius



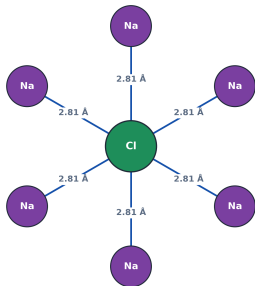
Nodes = atoms. Every ion in the faded, infinite lattice is one node.

Edges = bonds to neighbours *within a cutoff radius r only* (bold, $r \approx 2.9$ Å here).

For this Cl⁻, that cutoff catches exactly its 6 nearest Na⁺.

In the real pipeline, $r = 8$ Å and up to 12 neighbours are kept per atom – the idea is identical, just with more neighbours in view.

The abstraction: what the network sees



node = one atom · edge = one bond within the cutoff

Strip away the projection, the perspective, the rest of the crystal fading into the background – and this is what is left: one small, labelled graph.

No 3D coordinates are given to the network directly. Only:

- which atom each node is
- which pairs of atoms are connected
- how far apart each connected pair is

This is not a simplification of what CGCNN sees – it *is* what CGCNN sees. A crystal, to this network, is exactly this object.

What's inside a node

Each node must carry *what element it is*. Rather than learning this from scratch, every atom is looked up in a fixed table.

`atom_init.json`: one 92-dimensional vector per element, built once from tabulated properties – group, period, electronegativity, covalent radius, valence count – one-hot binned.

These numbers are **fixed, never learned** – the starting description of “what element is this”, identical for every atom of a given element in the dataset.

The network's first layer is a linear projection turning this fixed 92-dim description into the model's hidden width – the one place element identity becomes something *learned*.

Analogous to a lookup embedding table in NLP – except these starting vectors are physically meaningful, not randomly initialised.

Expanding the bond length

A bond has one obvious number: its length. Feeding that number in raw turns out to be a bad idea.

$$e_k(d) = \exp\left(-\frac{(d - \mu_k)^2}{\sigma^2}\right) \quad \mu_k = 0, 0.2, \dots, 8 \text{ \AA} \quad \sigma = 0.2 \text{ \AA}$$

The problem with one number

A single scalar forces the network to learn a sharply non-linear response from scratch – hard to fit, slow to converge.

What the expansion buys

A 2.31 Å bond lights up the basis functions near 2.3, barely touching the one at 5.0 – smooth and easy to learn, not a cliff.

This is a radial basis expansion – the same trick used throughout atomistic machine learning, not something specific to CGCNN.

Assembling one training example

For a crystal of N atoms, the graph handed to the network is exactly three tensors:

Atom features

$N \times 92$

one fixed row per
atom

Bond features

$N \times 12 \times 41$

Gaussian-expanded
distance to each of up
to 12 neighbours

Neighbour index

$N \times 12$

which row of the atom
table each neighbour
is

This triple, plus the true modulus in GPa, is one row of training data. The periodic neighbour search that builds it is the single slowest step in the whole pipeline – for 10,987 crystals it is computed once and cached.

Every crystal in this project, no matter how large or exotic, is reduced to exactly this shape before it ever reaches the network.

PART THREE

The Network Itself

How the graph turns into one number

In plain words, before the equations

"Each atom asks its neighbours: what are you, and how far away? Then it updates its own description based on the answers it hears."

That single round of asking-and-updating is called **message passing**, and it is the entire idea behind the network's convolution step. Do it once, and every atom's description reflects its immediate neighbours. Do it three times – as this network does – and every atom's description reflects everything happening within three bonds of it.

The next slide formalises exactly this sentence as an equation. Nothing new is being introduced – only made precise.

If the equations on the next slide feel abstract, come back to this sentence. That is all they are computing.

The gated convolution

For atom i with neighbours j , form the (self, neighbour, bond) triple for every edge, then update:

$$z_{ij} = v_i \oplus v_j \oplus e_{ij}$$

$$v_i \leftarrow v_i + \sum_j \sigma(z_{ij}W_f + b_f) \odot g(z_{ij}W_s + b_s)$$

$\oplus$ concatenation $\odot$ elementwise product σ sigmoid (the gate) g softplus (the message) v_i the running description of atom i

One shared linear layer processes every edge in the entire crystal identically. That sharing is what makes this a *convolution*: the same local rule applies everywhere, whether the crystal has 8 atoms or 200.

This one update, stacked three times, is the entire learned part of turning a graph into a description of the crystal.

Three design choices, and why

The gate $\sigma(\cdot)$

Decides, per bond, how much that neighbour contributes. A weak, distant bond can be down-weighted without being deleted from the graph.

Shared weights

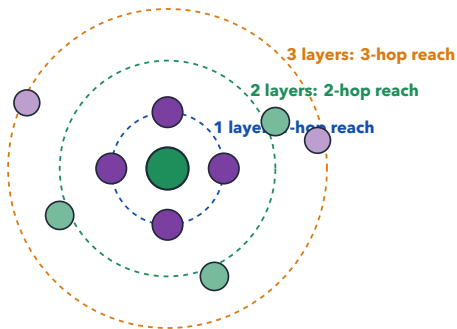
The same linear layer processes every edge. That is what makes this a convolution, and why it generalises across crystal sizes.

The residual, $v_i + (\cdot)$

Gradients flow straight through the stack, so early layers are not washed out by later ones as three layers compound.

Three convolution layers are used here, so every atom's final description reflects roughly a three-bond-hop neighbourhood.

Stacking layers grows the neighbourhood



Each additional layer lets information travel one more bond hop. After 3 layers, the centre atom's description is shaped by every atom within 3 bonds – with no global, whole-crystal operation ever computed.

More layers see further, at the cost of more computation and a risk of over-smoothing – which is why 3 is a deliberate choice, not a default.

From atoms to one number: pooling

$$V_c = \frac{1}{N} \sum_{i=1}^N v_i$$

Average, not sum, over every atom's final vector to get one description of the whole crystal.

A modulus is an **intensive** property – doubling the unit cell must not double the prediction.

A *sum* would make the output scale with however many atoms the CIF author happened to write down. The *mean* does not.

A small fully-connected head then turns that one 128-dimensional vector into a single number: the predicted modulus.

This is the one place the model has to actively resist a bias the data would otherwise hand it for free.

What we predict, and why its logarithm

Train on $\log_{10}(\text{modulus})$

Moduli here span 1-575 GPa. Under plain MSE on raw values, a 40 GPa error on a stiff crystal produces 100× more gradient than the same *relative* error on a soft one.

Exactly the wrong bias here

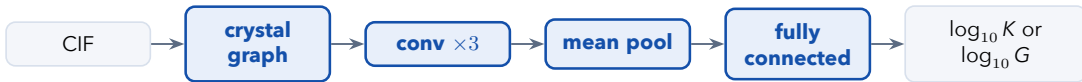
PINK screens for **ultralow** conductivity – the soft end of the range. A model that only cares about stiff crystals is worthless for this project's actual goal.

The logarithm converts “within a factor of x ” into “within a fixed distance” – so soft and stiff crystals carry equal weight during training. Normalisation statistics (mean, std) come from the training split only, so no information about held-out crystals leaks in.

This single transform is why the model is not simply best at predicting hard, boring materials.

16 • THE NETWORK

The full forward pass, start to finish



Every piece of this diagram has now been explained: how a crystal becomes a graph (Part 2), what the convolution and pooling steps do and why (this part), and why the target is a logarithm. Nothing else is hidden inside this network.

80,833 parameters in total – small by deep-learning standards, because the graph representation already does most of the work.

PART FOUR

Our Data and Training

What to train on, and how three models beat one

What to train on

1,213 CIFs – Materials Project structures, no labels. The **prediction** set.

10,987 matbench crystals
– DFT-computed K, G , the paper's own benchmark.
The **training** set.

Only 278 of the 1,213 appear in matbench – tempting to label just those.

The structures were never the limit – the labels were

Intersecting an unlabelled collection with a labelled benchmark discards 97% of the labels that exist. Invert the roles: train on all 10,987 matbench crystals, treat the 1,213 CIFs purely as a prediction set – 40× more training data, and what the paper itself does.

When labels are the scarce resource, never let an unlabelled collection define the training set.

Three models, averaged

Two networks trained on the same data with different random initialisations land in different minima. They **agree** about the signal and **disagree** about their own noise – so averaging keeps the first and partly cancels the second.

Average in log space

10 and 1000 GPa average to 100 GPa in log space, 505 GPa linearly. 100 is the defensible answer.

One shared split

Initialisation varies between the three models; the train/val/test split does not – or the ensemble's score is measured partly on data some member trained on.

Free uncertainty

Spread between the three predictions is a per-crystal confidence signal – large spread means the ensemble is guessing.

Ensembling typically buys 10-15% lower error. Ours gained 9% on bulk modulus, in that range.

PART FIVE

Results

How well it actually works

Model performance

Model	MAE $\log_{10}$ (GPa)	R^2	MAE (GPa)	Rel. error
Bulk, single model	0.0696	0.882	11.54	17.4%
► Bulk, 3-model ensemble	0.0630	0.901	10.20	15.6%
Shear, single model	0.0836	0.889	7.71	21.2%
► Shear, 3-model ensemble	0.0781	0.899	7.27	19.7%

Measured on 1,648 held-out crystals. All models share one train/validation/test split, so every row is directly comparable.

No overfitting

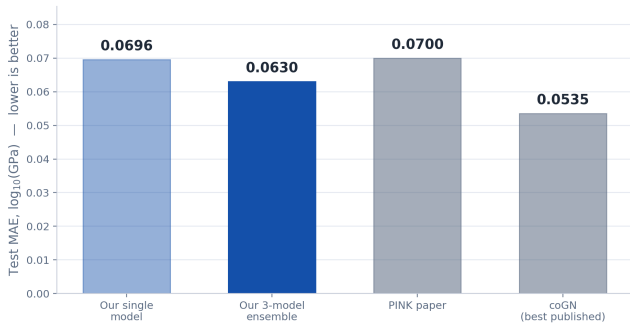
Train 0.0318 vs. test 0.0630 – ordinary generalisation error, not recall.

Ensembling gain

9% of the bulk-modulus error removed by averaging three initialisations.

20 • RESULTS

How we compare to the literature



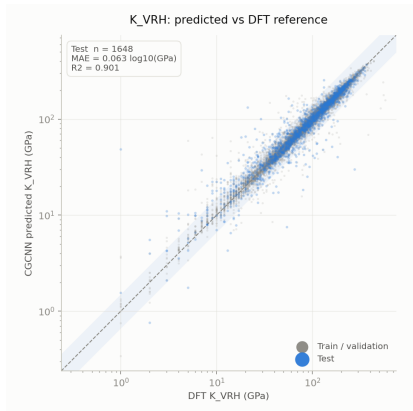
Reading this: our bulk ensemble slightly **beats** the paper it reproduces.

It sits between PINK and coGN – the best result ever published on this benchmark, with any architecture.

Same architecture, same training data as the paper. The gain comes from ensembling and a cosine learning-rate schedule.

Reproduction target met: 0.0630 against the paper's ≈ 0.070 .

Predicted vs. DFT --- bulk modulus



Every point is one crystal: DFT value on x , our prediction on y . A perfect model puts every point on the diagonal.

Points above the line are over-predictions; below, under-predictions.

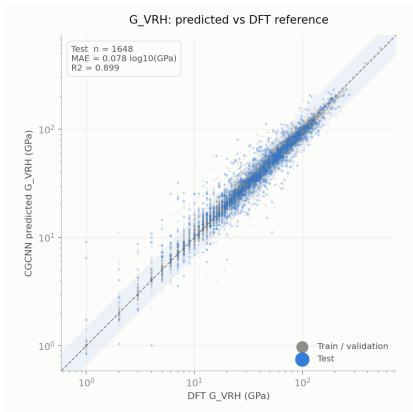
The shaded band marks “within a factor of 1.5” – a useful screening tolerance, where ranking matters more than the exact value.

Blue = the 1,648 held-out test crystals.

Grey = training data, shown for context.

$R^2 = 0.901$ – the model explains 0.901 of the variance in bulk modulus across the whole test set.

Predicted vs. DFT --- shear modulus



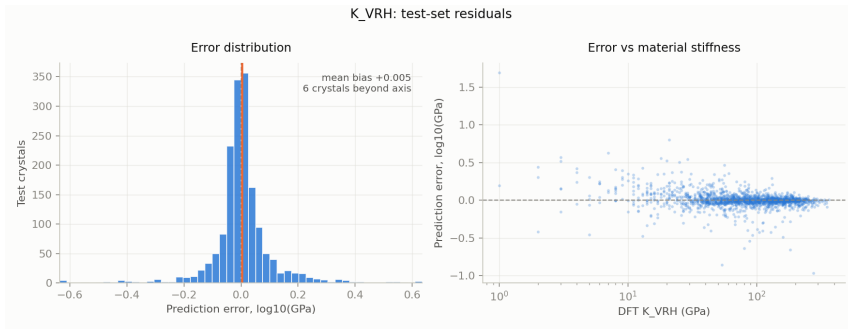
MAE 0.0781 against 0.0630 for bulk – consistently worse, for a physical reason.

Bulk modulus resists uniform compression, dominated by average bond stiffness – close to what a graph network reads off directly.

Shear modulus resists *shape change*, which depends on bond *directionality* and anisotropy the representation captures less directly.

The same ordering appears in the paper and across the matbench leaderboard – a property of the task, not of this implementation.

Where the errors live

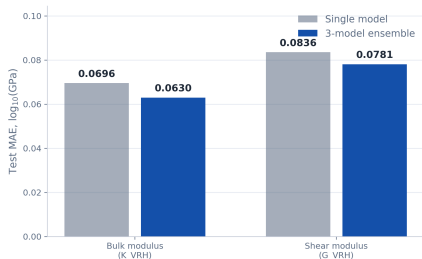


Left: unbiased. The error distribution centres on zero – the model does not systematically over- or under-predict.

Right: the honest weakness. Errors fan out below ~ 30 GPa – least reliable on soft materials, precisely the ultralow- κ regime PINK targets.

24 • RESULTS

Ensembling, and how it trains



Consistent gains on both targets – about 9% of the error, in line with the 10-15% typically expected.



Cosine annealing over 200 epochs. Train and validation track closely throughout – no divergence, so no overfitting.

How low can the error go?

Because the model trains on $\log_{10}$, MAE converts to a **multiplicative** error of $10^{\text{MAE}} - 1$.

MAE 0.0630 $\rightarrow$ within a factor of 1.156 $\rightarrow$ 15.6% typical error

A $\sim$ 2% relative error is not reachable on this task – by anyone

2% needs MAE = 0.0086 – about six times better than the best result ever published on this benchmark, with any architecture. The blocker is **label noise**, not model capacity: the targets are DFT-computed moduli, and DFT elastic constants themselves disagree with experiment by roughly 5–15%. A model cannot be more accurate than the labels it is fitted to – one reporting 2% here would be revealing a leak, not an achievement.

Realistic floor for this architecture: $\approx 0.06 \log_{10}$, about 15%. We are at 0.0630.

PART SIX

Wrapping Up

What we did differently from the paper

1. **Written from scratch** – graph construction, convolution, pooling, training loop: our own implementation, not a fork. No pre-trained weights are ever loaded.
2. **Ensembles, not single models** – three initialisations per target, averaged in log space, worth 9% of the error and a per-crystal uncertainty the paper lacks.
3. **Adam with cosine annealing**, not the original's SGD with step decay – a scheduled decay beats a reactive one on a fixed budget.
4. **Provenance on every prediction** – each row records whether the model trained on that crystal, so recall is never mistaken for generalisation.
5. **A documented error floor** – reporting both the multiplicative and range-normalised errors, not whichever flatters more.

Deliberately identical: the architecture, the training data, the log target, the split ratio. In a reproduction, differences should be few and intentional.

The deliverable

`results/pink_moduli_predictions.csv`

Bulk and shear modulus for all 1,213 crystals – with Pugh ratio, DFT reference where one exists, per-crystal ensemble uncertainty, and a provenance tag. This is the input the κ_L stage consumes.

935

crystals genuinely
unseen

278

in matbench,
tagged as recall

42

in the held-out test
split

A built-in correctness check: on those 42 test-provenance crystals the error matches the benchmark test error exactly – the provenance mapping is doing its job.

Summary

- A crystal is a **graph**: atoms as nodes, bonds within a cutoff as edges – shown concretely in real NaCl.
- A **gated convolution** lets each atom update its description from its neighbours; stacked three times, information travels three bonds.
- **Mean pooling** and a **log-scale target** are both deliberate choices, not defaults – one respects an intensive property, the other prevents the model from ignoring soft materials.
- The scarce resource was **labels, not structures** – training on all 10,987 matbench crystals rather than the 278-crystal overlap more than halved our error.
- **Ensembling** three models bought a further 9%, landing at $0.0630 \log_{10}(\text{GPa})$ for bulk modulus – matching, and slightly beating, the paper this work reproduces.

Thank you.